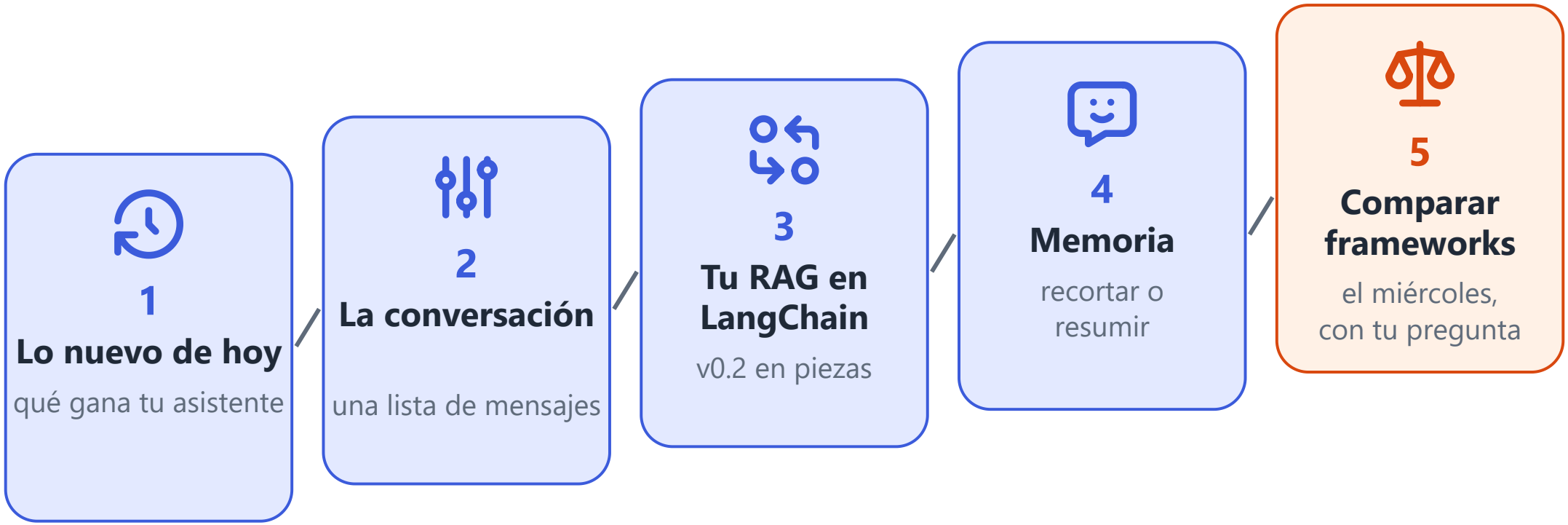
LangChain y memoria

M5: v0.3, LangChain y memoria



De código a mano a piezas armadas



Venís de v0.2, armado a mano. Hoy lo pasás a piezas que encajan y le das memoria.

Hoy cada ejemplo corre en vivo: cuaderno de la clase 4.

Primero solo, después lo discutimos

- Control de lectura individual, 10 min
- Discusión de las respuestas, 5 min

Sin asistentes de IA. 5 preguntas sobre frameworks, memoria y las lecturas. En la discusión, las mismas.

¿Para qué te sirve? Un asistente que sigue la charla

Dónde	Pasa	Lo resuelve
BI	«¿Y por región?» tras el churn	Memoria
Soporte	Cada cliente, su charla	Un hilo por cliente
Equipo	Cambiar Gemini por OpenAI	El framework
Producto	Sumarle herramientas	Piezas que encajan

Para el modelo, una conversación es una lista

```
system     Respondé solo con el contexto y citá el documento.
user       ¿Cómo se calcula el churn?
assistant  Bajas del mes / clientes al inicio [churn].
user       ¿Y por región?
```

- Cada mensaje tiene un rol: instrucciones, vos, el asistente.
- «¿Y por región?» solo se entiende leyendo lo de arriba.

La API no recuerda nada entre pedidos

```
sin = [("system", SISTEMA), ("human", "¿Cómo me llamo?")]
con = [("system", SISTEMA), ("human", "Me llamo Ana."), ("ai", "¡Hola, Ana!"),
       ("human", "¿Cómo me llamo?")]
modelo.invoke(sin).text  # «Todavía no me dijiste tu nombre, ¡así que no lo sé!»
modelo.invoke(con).text  # «Te llamás Ana.»
```

- Memoria = tu programa reenvía la charla en cada pedido.
- ¿Qué pasa si dos personas la usan a la vez?

Cada conversación guarda solo lo suyo

Hilo A
2 preguntas
4 mensajes guardados

≠

Hilo B
1 pregunta
2 mensajes guardados

- **Hilo** = una conversación con su historia: como filtrar los mensajes por hilo.
- ¿Eso es memoria?

El estado: lo que el hilo guarda en cada paso

```
config = {"configurable": {"thread_id": "A"}}
estado = asistente.get_state(config)
[m.type for m in estado.values["messages"]]  # ['human', 'ai', 'human', 'ai']
```

- **Estado** = la foto de la charla: hoy, la lista de mensajes.
- El *checkpointer* la guarda después de cada paso.

Memoria corta y memoria larga

Corta: dentro de una charla
el estado del hilo
***checkpointer*, en tu v0.3**

≠

Larga: entre charlas
datos del usuario: «prefiere tablas»
***store*, por usuario**

Como los filtros de esta sesión y el perfil guardado del usuario.

Dos corridas: el seguimiento acertó una vez

```
corrida 1  Hilo A (seguimiento): NO ENCONTRADO
           Hilo B:               Si el proceso se reinicia, todos los puntos de
                                 control … se pierden [langgraph__persistence].
corrida 2  Hilo A (seguimiento): Si se utiliza MemorySaver o InMemorySaver,
                                 todos los puntos de control se pierden…
           Hilo B:               Cuando el proceso se reinicia, … se pierden…
las dos    Mensajes guardados → hilo A: 4 · hilo B: 2
```

- Probalo: cuaderno en vivo, sección 5
- Buscó solo con «¿y qué pasa con eso?»: hoy lo arreglás.

LangChain: piezas que armás vos

Pieza	Qué hace	En tu v0.3
Modelo	Redacta	`cargar_modelo()`
Prompt y cadena (LCEL)	Arman el pedido, paso a paso	`reformular`
Middleware	Corre antes del modelo	`con_contexto`
Agente	Mensajes → respuesta	`create_agent`
Checkpointer	Guarda cada hilo	`InMemorySaver`

Una función que corre antes del modelo

```
@dynamic_prompt  # la @: LangChain la corre antes de cada llamada al modelo
def con_contexto(pedido: ModelRequest) -> str:
    hits = buscar(pedido.messages[-1].text)  # tu búsqueda de v0.2
    contexto = "\n\n".join(f"[{h['doc']}]\n{h['texto']}" for h in hits)
    return f"{SISTEMA}\n\nContexto:\n{contexto}"
asistente = create_agent(cargar_modelo(), tools=[], middleware=[con_contexto],
                         checkpointer=InMemorySaver())  # guarda cada hilo
```

Busca, arma el contexto y guarda cada hilo. En vivo: sección 1.

LCEL: reescribí el seguimiento antes de buscar

```
reformular = ChatPromptTemplate.from_messages([
    ("system", "Reescribí la última pregunta para que se entienda sola."),
    ("human", "Charla:\n{charla}\n\nÚltima pregunta: {pregunta}"),
]) | cargar_modelo() | StrOutputParser()  # cada | pasa su salida
nueva = reformular.invoke({"charla": charla, "pregunta": "¿Y por región?"})
```

- Entra «¿Y por región?»; sale «¿Cómo se calcula el churn por región?».
- Como los pasos de Power Query: cada uno recibe la salida del anterior.

LangChain no empeoró la suite

v0.2, Chroma
hit 10/11

=

v0.3, LangChain
hit 10/11, mismos fragmentos

Dos corridas (15/10 y 17/10): **sin regresión**. En vivo: sección 3.

Taller: el seguimiento que se pierde (50 min)

1. **Corré** `v03_memoria.py` y `medir.py`: mismo hit
2. **Tu charla:** ¿qué `fuentes` trae el seguimiento?
3. **Reformulá** y mirá de nuevo

Práctica M5, arranque y «En clase», pasos 1 a 3.

Cada turno reenvía toda la charla

Turno	Mensajes que viajan	Con límite
1	1	1
2	3	3
5	9	3
10	19	3

- Pagás tokens por toda la historia, en cada pedido.
- El hilo guarda todo; el límite decide qué ve el modelo.

Recortar o resumir: qué ve el modelo

	Recortar	Resumir
Ve	Los últimos mensajes	Un resumen y los últimos
Cuesta	Parejo	Una llamada más
Pierde	Lo viejo, entero	Detalles
En LangChain	`trim_messages`	`SummarizationMiddleware`

El hilo guarda todo igual: el límite decide qué ve el modelo.

¿Cuánta memoria para tu caso?

Caso	Conviene
Soporte: preguntas cortas	Los últimos mensajes
Un análisis largo con un cliente	Resumir
Datos sensibles	No guardar: un hilo nuevo por pedido

Más memoria responde mejor los seguimientos y cuesta más.

Taller: memoria con límite (35 min)

1. **Aislamiento:** B no guarda lo de A.
2. **Charla larga:** cinco turnos, mirá los tokens.
3. **Recortá** con `trim_messages`: ¿qué olvida?

Práctica M5, pasos 4 a 6. Para casa: ¿recortar o resumir?

LlamaIndex hace solo lo que en v0.2 escribiste



La demo muestra qué decidió por vos.

El mismo RAG en LlamaIndex son tres líneas

```
docs = SimpleDirectoryReader("corpus").load_data()          # leer
indice = VectorStoreIndex.from_documents(docs)              # partir y vectorizar
r = indice.as_query_engine(similarity_top_k=4).query(p)     # buscar y responder
```

- Tu `v02_rag.py` hace lo mismo a la vista.
- Ojo: de fábrica usan OpenAI y leen todo `corpus/`, licencia incluida. La demo antes les dice qué modelo y qué archivos usar.

LlamaIndex respondió largo y sin citar

```
Pregunta: ¿Qué hace interrupt en LangGraph?

── Tu v0.2 · responder (2.3 s)
`interrupt` pausa la ejecución de un grafo en un nodo … y espera una entrada
externa antes de continuar [langgraph__interrupts]. Al llamarlo, … guarda el estado

── LlamaIndex (35.3 s)
Basándose en la información provista, la función `interrupt()` … permite pausar
… 1. **Pausa la ejecución** … 2. **Guarda el estado** … 3. … 4. …   (sin citas)
fuentes: […] · fragmentos: 50
```

- Probalo: cuaderno en vivo de la clase 5, sección 1
- Los 35 s incluyen rehacer el índice. v0.2 lo reusa.

Lo que el framework decide por vos

	v0.2	LlamaIndex, de fábrica
Fragmento	1000 caracteres	1024 tokens
k	4	2
Prompt	Tu `SISTEMA` : citar	El suyo, oculto
Índice	En `.chroma`	Se rehace cada vez

Todo se cambia, pero primero hay que saber que existe.

Comparemos con una pregunta tuya

- Dictá una pregunta de tu tema: la corro en los dos.
- Anotá de cada uno: largo, si cita, tiempo y fuentes.
- ¿Cuál dejarías en tu proyecto, y por qué?

Repaso: hoy te llevás tres ideas



Framework

piezas armadas, pasos escondidos



Memoria

reenviar la conversación



Medir

igual, con cualquier librería

Un hilo por persona: cada conversación guarda solo lo suyo.

¿Qué ya no podrías explicar?

Nombrá una parte de tu sistema que ahora hace LangChain y que no sabrías explicar sin leer su código.

Anotá tu **confianza (1 a 4)**.